

```

1 =====
2 CLAUDE-LIKE-APP – FULL OPEN-SOURCE ARCHITECTURE MAP
3 =====
4 Builds on top of the zero-trust-ai-app foundation already built (Keycloak +
5 SPIRE + FastAPI). This document maps every remaining capability to a specific,
6 currently-maintained, self-hostable open-source tool, with the reasoning
7 behind each pick and how the pieces connect.
8
9 =====
10 1. LLM GATEWAY – routing, fallback, cost tracking, rate limits, retries
11 =====
12 TOOL: LiteLLM Proxy (self-hosted, open source)
13
14 Why: it's the de-facto open-source gateway for exactly this job – one
15 OpenAI-compatible endpoint in front of 100+ providers (Claude + OpenAI covered
16 natively), with load balancing, automatic fallback between providers,
17 per-key/per-team budget enforcement, RPM/TPM rate limiting, and full spend
18 logging, all configured in one YAML file with no code changes in your app.
19
20 What it gives you directly, out of the box:
21   - Routing between Claude and OpenAI (and swapping providers on outage)
22   - Automatic fallback groups (e.g. Claude primary -> GPT-4o fallback)
23   - Exponential-backoff retries built in (allowed_fails + cooldown config)
24   - Per-key budget caps and usage-based rate limits
25   - Cost tracking per model/user/team, exportable to your observability layer
26   - Logs to Langfuse / OpenTelemetry natively (see Section 8)
27
28 Where it sits: every LLM call from your backend goes through LiteLLM instead
29 of hitting api.anthropic.com or api.openai.com directly. Your backend just
30 points its OpenAI-SDK-compatible client at http://litellm:4000/v1.
31
32 Config sketch (docker-compose service + config.yaml):
33   model_list:
34     - model_name: claude-primary
35       litellm_params:
36         model: anthropic/claude-sonnet-5
37         api_key: os.environ/ANTHROPIC_API_KEY
38     - model_name: gpt-fallback
39       litellm_params:
40         model: openai/gpt-4o
41         api_key: os.environ/OPENAI_API_KEY
42   router_settings:
43     fallbacks: [{"claude-primary": ["gpt-fallback"]}]}
44     allowed_fails: 3
45     cooldown_time: 30

```

```

46
47 =====
48 2. MEMORY – short-term, long-term, episodic, semantic
49 =====
50 SHORT-TERM (working/session memory):
51 TOOL: Redis
52   - Holds the active conversation buffer, in-flight tool call state, and
53     per-session scratch data. Fast, ubiquitous, and doubles as your caching
54     layer (Section 9) and rate-limit counter store for LiteLLM.
55
56 LONG-TERM (episodic + semantic, persisted across sessions):
57 TOOL: Mem0 (self-hosted, open source) – recommended starting point
58   - Lightweight, vector-based memory layer purpose-built for this: extracts
59     structured facts from conversations and retrieves by semantic
60     similarity, with user/session/agent-scoped memory out of the box.
61     Far smaller token/latency footprint than graph-based alternatives,
62     which matters for an interactive chat app.
63   - Runs self-hosted against your own Postgres + a vector store (Section 3).
64
65 ALTERNATIVE if you need temporal/graph reasoning ("what did I tell you
66 three sessions ago", contradiction resolution over time):
67 TOOL: Graphiti (the open-source engine behind Zep, Apache 2.0)
68   - Temporal knowledge graph, self-hostable on Neo4j, FalkorDB, or Kuzu.
69     Stronger for multi-hop, entity-centric, time-aware recall; heavier
70     (larger memory footprint, slower retrieval) than Mem0 in practice.
71
72 Recommendation: start with Mem0 for episodic + semantic memory (fast,
73 simple, self-hosted, low footprint). Only reach for Graphiti/Zep if you
74 specifically need "how did this fact change over time" reasoning that Mem0's
75 flat vector recall can't express.
76
77 =====
78 3. DATABASE + VECTOR STORE
79 =====
80 RELATIONAL: PostgreSQL
81   - Already in your stack (backing Keycloak). Reuse it (separate schema/DB)
82     for: conversation metadata, user/session records, Mem0's structured
83     fact store, RAG document metadata, and Langfuse's own backing store.
84
85 VECTOR STORE (for RAG + Mem0 embeddings):
86 TOOL: Qdrant or pgvector (Postgres extension)
87   - pgvector: simplest – one fewer service to run, since it's just an
88     extension on the Postgres you already have. Good up to moderate scale.
89   - Qdrant: dedicated vector DB, better at scale and richer filtering.
90 Recommendation: start with pgvector (fewer moving parts), migrate to

```

91 Qdrant only if you hit real scale/latency limits.

92

93 =====

94 4. RAG PIPELINE + RAG EVALS

95 =====

96 RAG PIPELINE: your existing stack already covers this well –

97 LlamaIndex or LangChain for chunking/retrieval orchestration, embeddings

98 via an open embedding model (or Claude/OpenAI embeddings through LiteLLM),

99 pgvector/Qdrant for storage.

100

101 RAG EVALS:

102 TOOL: Ragas (open source) – you already listed this

103 - Purpose-built RAG eval metrics: faithfulness, answer relevancy, context

104 precision/recall. Run as offline evaluation jobs against a golden dataset.

105 TOOL: DeepEval (open source) – also already in your stack

106 - Broader LLM eval framework (hallucination, bias, toxicity, custom

107 metrics), integrates with pytest for CI-style eval gating.

108 Both plug directly into Langfuse's dataset/experiment features (Section 8),

109 so eval results show up alongside your production traces.

110

111 =====

112 5. MULTI-AGENT ORCHESTRATION, TOOL CALLING, MCP

113 =====

114 TOOL: LangGraph (open source) – you already have this in your stack

115 - Handles multi-agent orchestration, tool-call routing, and stateful

116 graphs. If you go this route, LangMem (built on LangGraph) becomes a

117 natural fit for Section 2's memory layer instead of Mem0 – trade-off is

118 ecosystem lock-in vs. one less new dependency.

119

120 MCP (Model Context Protocol):

121 - Expose your tools (the ones behind Act 2/3's tools:invoke scope) as MCP

122 servers so any MCP-compatible agent/client (including Claude itself) can

123 call them through a standard interface, instead of bespoke function-

124 calling schemas per tool. This is exactly where your zero-trust Act 3

125 token exchange plugs in: the MCP server enforces the scoped token before

126 executing any tool call.

127

128 =====

129 6. SANDBOX – untrusted code execution

130 =====

131 IMPORTANT UPDATE: Daytona (in your existing stack) moved its production

132 codebase to closed source in June 2026 – the open-source repo is now archived

133 and unmaintained. For a fully open-source, self-hosted requirement, it's no

134 longer the right pick.

135

```
136 RECOMMENDED INSTEAD:
137 TOOL: E2B (open-source self-host path) or OpenSandbox (CNCF-hosted, org-
138 independent, ~11.5K stars, ships persistence + Firecracker/Kata isolation)
139   - Both use Firecracker microVMs – genuine hardware-level isolation (own
140     kernel per sandbox), the right bar for running arbitrary agent-generated
141     code, not just container-level (shared-kernel) isolation.
142   - Self-hosting E2B OSS is a real infrastructure project (Firecracker +
143     Nomad/Consul or similar) – budget real ops time for this, it is not a
144     `docker run`.
145
146 LIGHTER-WEIGHT ALTERNATIVE if full microVM isolation is overkill for your
147 threat model:
148 TOOL: gVisor + runc
149   - User-space kernel intercepting syscalls before they reach the host –
150     weaker boundary than Firecracker but far simpler to operate, and what
151     Daytona itself used under the hood.
152
153 Either way, the sandbox should sit behind the same Act 3 token-exchange gate
154 as any other tool: the agent's scoped token authorizes "may execute code",
155 the sandbox itself never trusts a request without it.
156
157 =====
158 7. SECURITY / ZERO TRUST
159 =====
160 Already built – see zero-trust-ai-app.txt. Recap of how it plugs into
161 everything above:
162   - Every service in this architecture (LiteLLM, Mem0, the sandbox, MCP
163     tool servers) sits behind FastAPI's role-based authorization (Act 2).
164   - Any inter-service call that isn't the human's own request (agent calling
165     a tool, agent calling the sandbox, agent calling an MCP server) goes
166     through the Act 3 token-exchange pattern: SPIRE-issued workload identity
167     -> Keycloak token exchange -> narrowly scoped token -> the actual call.
168   - This means adding a new tool or service to the system means adding a new
169     realm role + SPIRE workload registration, not a new bespoke auth scheme.
170
171 =====
172 8. OBSERVABILITY, TRACING, MONITORING, COST
173 =====
174 TOOL: Langfuse (self-hosted, open source)
175   - This is the single tool that covers tracing, evals, prompt management,
176     cost tracking, and metrics in one open-source platform, specifically
177     built for LLM/agent applications (not generic APM retrofitted).
178   - Nested trace view for multi-step agent runs: see every LLM call, tool
179     call, retrieval step, and retry as parent/child spans in one trace.
180   - Natively integrates with LiteLLM (Section 1) and OpenTelemetry, so cost
```

```
181     and token usage from your gateway flows straight into Langfuse dashboards.
182 - Datasets + experiments feature is where Ragas/DeepEval eval runs
183 (Section 4) get tracked over time, alongside production traces.
184
185 GENERAL INFRA MONITORING (uptime, latency, resource usage – not LLM-specific):
186 TOOL: Prometheus + Grafana
187 - Standard self-hosted stack for infra-level metrics (container health,
188   Postgres/Redis metrics, request latency at the FastAPI layer). Langfuse
189   covers the LLM-specific layer; Prometheus/Grafana covers everything else.
190
191 =====
192 9. CACHING
193 =====
194 TOOL: Redis (same instance as Section 2's short-term memory)
195 - Two layers of caching worth setting up:
196   1. Exact-match response cache (same prompt -> same cached completion),
197     keyed by a hash of the request – cheap, catches repeated queries.
198   2. Semantic cache (GPTCache, open source, works with Redis as the vector
199     backend) – catches near-duplicate queries via embedding similarity,
200     not just exact string matches. Bigger win for RAG-heavy workloads
201     where users phrase the same question differently.
202
203 =====
204 END-TO-END DATA FLOW (how a single request touches every layer)
205 =====
206   1. Alice sends a message -> FastAPI verifies her JWT (Act 1) and role (Act 2)
207   2. Backend checks Redis cache (exact + semantic) -> cache hit returns fast
208   3. On a miss: Mem0 is queried for relevant long-term memory (episodic +
209     semantic facts about Alice) to enrich context
210   4. If the query needs retrieval: RAG pipeline hits pgvector/Qdrant
211   5. Assembled prompt goes to LiteLLM -> routes to Claude (or falls back to
212     OpenAI on outage/rate-limit), with retries and budget enforcement
213   6. If the model requests a tool call: backend performs Act 3 token
214     exchange (SPIRE SVID -> Keycloak -> scoped token) before calling the
215     MCP tool server or sandbox
216   7. Every step above emits a span to Langfuse – the whole request is one
217     traceable, cost-attributed, evaluable unit
218   8. Response streams back to Alice; Mem0 asynchronously extracts any new
219     facts worth remembering for next time
220
221 =====
222 WHAT TO BUILD FIRST – suggested order
223 =====
224 Given everything above depends on the zero-trust foundation already in
225 place, a sensible build order is:
```

226
227
228
229
230
231
232
233
234
235
236
237
238
239

1. LiteLLM gateway (Section 1) – unblocks real Claude + OpenAI calls through your existing backend with minimal new infra
2. Redis caching + short-term memory (Sections 2, 9) – immediate latency and cost win, low complexity
3. Mem0 long-term memory (Section 2) – the "remembers you" feature
4. RAG + pgvector (Sections 3, 4) – knowledge retrieval
5. Langfuse observability (Section 8) – wire this in early rather than late; retrofitting tracing onto an already-complex system is painful
6. MCP tool servers + sandbox (Sections 5, 6) – the highest-complexity, highest-payoff piece, and the one most directly secured by your existing Act 3 token exchange

=====